



Metaheuristics for the traveling salesman problem with pickups, deliveries and handling costs

Güneş Erdoğan^{a,*}, Maria Battarra^b, Gilbert Laporte^c, Daniele Vigo^d

^a Industrial Engineering Department, Özyeğin University, Kuşbakışı Cad. 2, 34662 Altunizade, İstanbul, Turkey

^b School of Mathematics, University of Southampton, Southampton SO17 1BJ, UK

^c Canada Research Chair in Distribution Management, HEC Montréal, 3000 chemin de la Côte-Sainte-Catherine, Montreal, Canada H3T 2A7

^d DEIS, University of Bologna, via Venezia 52, 47521 Cesena, Italy

ARTICLE INFO

Available online 22 July 2011

Keywords:

Traveling salesman problem

Handling cost

Pickup and delivery

Metaheuristics

ABSTRACT

This paper studies the Traveling Salesman Problem with Pickups, Deliveries, and Handling Costs. The subproblem of minimizing the handling cost for a fixed route is analyzed in detail. It is solved by means of an exact dynamic programming algorithm with quadratic complexity and by an approximate linear time algorithm. Three metaheuristics integrating these solution methods are developed. These are based on tabu search, iterated local search and iterated tabu search. The three heuristics are tested and compared on instances adapted from the related literature. The results show that the combination of tabu search and exact dynamic programming performs the best, but using the approximate linear time algorithm considerably decreases the CPU time at the cost of slightly worse solutions.

© 2011 Published by Elsevier Ltd.

1. Introduction

The *Traveling Salesman Problem with Pickups, Deliveries and Handling Costs* (TSPPD-H), recently introduced by Battarra et al. [1], is a generalization of the *Single Vehicle Pickup and Delivery Problem with combined demands* (SVPDP-P&D) (see [2]). As in the SVPDP-P&D, the aim is to design an optimal tour through the depot and a set of customers, each of which requires a pickup service, a delivery service, or both. The vehicle leaves the depot carrying all the deliveries, visits each customer once, and returns to the depot carrying all pickups, without ever exceeding its capacity.

In the TSPPD-H the vehicle is represented as a single stack in which the commodities can be loaded and unloaded only from the rear. This vehicle configuration is commonly encountered in real world applications. The vehicle is a stack whenever the goods have one dimension comparable to the truck width. In Battarra et al. [1], the collection and delivery of damaged and undamaged bicycles in public bicycle sharing systems is presented as an example of TSPPD-H; we also mention the problem of delivering calves to farms and collecting mature cows [3]. Note that under the assumption of a single stack, pickup commodities may obstruct the delivery operations.

It is common practice to either disregard the handling operations and leave the driver free to decide about them, or to impose specific tour shapes [4,5] to handle the obstruction issue. For example, by forcing some or all delivery operations to take place

at the beginning of the tour in order to create some empty space on board. However, non-Hamiltonian tours can increase routing costs substantially. An example of LIFO application is the auto-carrier transportation problem presented in Dell'Amico et al. [6], in which it is shown that a significant improvement in routing cost could be obtained by dropping the LIFO constraint and allowing handling operations.

The TSPPD-H consists of determining an Hamiltonian tour in which the commodities on board are possibly rearranged at the customer locations. The objective is to minimize the sum of handling and routing costs, the handling cost being the time spent in rearranging the load on board in the vehicle, and the routing cost the time required to actually perform the Hamiltonian tour. The problem becomes particularly interesting whenever the handling cost is comparable to the routing cost and a tradeoff between handling and routing can be envisaged. In fact, if the routing cost is significantly higher than the handling cost, the problem can be solved efficiently as a SVPDP-P&D and then by optimizing the handling decisions based on the routing decisions. On the other hand, if the handling cost is significantly higher than the routing cost, the problem can be solved as a *Double-Path Single Vehicle Pick and Delivery Problem* [4], or possibly by using two vehicles, one for the pickup and one for the delivery service.

There is a growing interest in SVPDP-P&Ds involving handling decisions. Variants of the Traveling Salesman Problem with Pickups, Deliveries with LIFO loading [7,8], FIFO loading [9] and multiple stacks [10,11] have been recently studied, as well as Vehicle Routing Problems including loading issues [12]. We refer the reader to the surveys by Iori and Martello [13] and Laporte [14]. All applications pertaining to the SVPDP-P&Ds with rear-loading are valid for the

* Corresponding author.

E-mail address: gunes.erdogan@ozyegin.edu.tr (G. Erdoğan).

TSPPD-H if the vehicle load can be rearranged, and hence there exists a potential for cost reduction in these problems.

Battarra et al. [1] have shown that the optimal loading decisions for a given tour can be determined in polynomial time. However, these decisions cannot easily be implemented in practice since they may require an excessive amount of handling. Hence, three elementary loading policies were proposed in order to simplify the handling operations. Branch-and-cut algorithms were developed for each policy. The results highlighted the relative importance of handling costs with respect to routing costs. Policies 1 and 2 (in which the pickup commodities are located either in the rear or in the front of the vehicle at each customer, respectively) are very easy to implement, but compromise the overall solution quality. On the other hand, Policy 3 achieves high quality results. This policy consists of deciding either to place, at each customer location, the pickup commodities in the rear or in the front of the vehicle. Policy 3 is simpler to implement than the optimal policy for the driver because the pickup and delivery commodities on board exhibit a contiguous pattern. Moreover, it is more efficient than the first two policies, because it avoids long queues of pickup commodities at the rear, as well as too frequent handling of delivery commodities.

The TSPPD-H, being a generalization of the SVPDP-P&D, is an *NP-hard* problem which combines the complexity of routing and scheduling decisions. Although promising results were reported under Policy 3, exact algorithms were only able to solve small-size instances (with up to 25 customers). In this paper, we therefore develop and compare powerful metaheuristics for the TSPPD-H. Our goal is to determine high quality solutions for large size instances within limited computing time. To this end, we first study the handling cost subproblem in greater depth, we next analyze possible neighborhoods and their evaluation, and we finally describe the three metaheuristics.

The remainder of the paper is organized as follows. Section 2 introduces the notation and describes the algorithms developed for the handling cost subproblem. Local search operators are analyzed in Section 3. Section 4 describes the proposed metaheuristic algorithms. Computational results are presented in Section 5, followed by conclusions in Section 6.

2. The handling cost subproblem

Let $G=(V, A)$ be a complete directed graph where $V=\{0, \dots, n\}$ is the vertex set and A is the arc set. Vertex 0 is the depot, whereas $V_c = V \setminus \{0\}$ are the customers. Each arc $(i, j) \in A$ is associated with a travel time c_{ij} , and each customer $i \in V_c$ are associated with α_i delivery commodity units and β_i pickup commodity units. There are $\alpha_0 = \sum_{i \in V_c} \alpha_i$ delivery commodity units originating from the depot, and $\beta_0 = \sum_{i \in V_c} \beta_i$ pickup commodity units destined to the depot. We assume both commodities have the same unit dimensions (for example boxes, pallets, or containers of the same goods). We denote the delivery and pickup commodities as type a and b , respectively. The vehicle capacity is Q units.

As previously stated, the vehicle follows a Last-In-First-Out (LIFO) loading policy [15]. When visiting a customer, the type b commodities obstructing the delivery commodities have to be unloaded. The time spent in unloading and reloading a type b commodity on board is equal to h_b . In addition, type a commodities may be unloaded and reloaded in order to place pickup commodities in a more suitable position. The time spent to unload and reload a type a commodity is equal to h_a . We denote these unloading and reloading activities as *additional operations*. The handling operations strictly required to perform delivery and pickup are constant and unavoidable, and are therefore not taken into account in the handling cost evaluation. We refer to Battarra

et al. [1] for an exact mixed integer linear programming formulation of the problem.

The optimal solution for the TSPPD-H is a Hamiltonian tour on G which minimizes the sum of routing costs and additional operation costs. As a consequence, in a local search algorithm, moves in the neighborhood of a solution have to be evaluated considering these two aspects. The extra travel time cost can be simply computed as in the routing literature: the cost of the newly introduced arcs is added to the previous routing cost, whereas the cost of the deleted arcs is subtracted. This extra travel time computation is a constant time complexity operation, so it does not interfere with the algorithm speed. On the other hand, computing the handling cost variation is more difficult. For example, every time a customer is relocated in the tour, the overall handling cost has to be computed from scratch. The handling cost computation can be a time consuming task: any improvement to speed up this task should then result in an immediate reduction of the overall computing time.

In the remainder of this section we propose two ways of solving the handling cost subproblem. The first one is an exact dynamic programming algorithm with quadratic time and space complexity, and the second one is a linear time heuristic based on the solution of a special case of the handling cost subproblem.

2.1. Dynamic programming algorithm

Given a tour, in order to compute an exact optimal solution to the handling cost subproblem under Policy 3, we need to decide between placing the pickup commodities at the rear (Policy 1) or at the front (Policy 2) of the vehicle at every customer location. The cost of each decision is based on the current configuration of the commodities on board. This naturally leads to the idea of using the number of pickup and delivery boxes as state variables. However, this approach inevitably leads to a pseudopolynomial complexity algorithm. We now provide a genuinely polynomial dynamic programming algorithm to solve the handling cost subproblem.

For the sake of brevity and clarity, assume that we have renumbered the customers according to their order in the given tour. Let $f(i)$ denote the optimal cost of handling customers $i+1$ to n , given that Policy 2 has been applied at customer i . Furthermore, let p_{ij} denote the cost of applying Policy 1 at customers $i+1$ to $j-1$, and Policy 2 at customer j :

$$(DP) \quad f(i) = \min_{j \in \{i+1, n\}} \{p_{ij} + f(j)\}, \quad \forall i \in \{0, \dots, n-1\}, \quad (1)$$

$$f(n) = 0. \quad (2)$$

In DP, there are n states, each of which requires $O(n)$ computing time, which yields a complexity of $O(n^2)$. However, the application of DP requires the values of $p_{ij}, \forall i, j \in \{1, \dots, n\}$ to be computed beforehand. This is achieved through Algorithm 2.1, which also has an $O(n^2)$ complexity. In the pseudo-code, α' and β' denote the number of delivery and pickup commodities on board, respectively, and θ is the cumulative cost of applying Policy 1.

Algorithm 2.1. $P(\alpha, \beta, h_a, h_b)$

```

for  $i=0$  to  $(n-1)$ 
   $\beta' = 0$ ;  $\theta = 0$ ; // Policy 2 has been applied
  // compute the remaining number of delivery items on board
   $\alpha' = \alpha_0$ ;
  for  $j=1$  to  $i$ 
     $\alpha' = \alpha' - \alpha_j$ ;
  for  $j=i+1$  to  $n$ 
     $\alpha' = \alpha' - \alpha_j$ ; // deliver, no matter Policy 1 or 2
    // cost of Policy 1 up to this point plus the cost of Policy 2
    if  $(\beta' + \beta_j > 0)$   $p_{ij} = \theta + h_a \alpha' + h_b \beta'$ 
    else  $p_{ij} = \theta$ 

```

```
// Policy 1
if ( $\alpha_j > 0$ )  $\theta = \theta + h_b \beta'$ 
// pickup as if Policy 1 is used
 $\beta' = \beta' + \beta_j$ 
```

In order to solve the handling cost subproblem under Policy 3, Algorithm 2.1 and DP should be applied sequentially. Since both algorithms have $O(n^2)$ time and space complexities, this is also the case for the overall algorithm. It is well known that classical local search operators (*relocate*, *2-opt*, and *swap*) require the evaluation of $O(n^2)$ moves in the neighborhood of a solution. Consequently, the complexity of any local search algorithm for TSPPD-H becomes $O(n^4)$ at each iteration, which may result in prohibitively large CPU times for large instances. To avoid this problem, we next focus on designing a linear time heuristic.

2.2. Linear time heuristic

We now consider a special case of the handling cost subproblem, in which all customers supply $\bar{\alpha}$ delivery commodities, require $\bar{\beta}$ pickup commodities, and we apply Policy 2 a total of p times ($p \in \{1, \dots, n\}$). Let $k_1, \dots, k_p$ denote the customer count after which we (re)apply Policy 2, i.e., the first application of Policy 2 occurs at customer k_1 , the second one at customer $k_1 + k_2$, and the last one at $\sum_{i=1, \dots, p} k_i$. For $i = 1, \dots, p-1$, using Policy 1 for $k_i - 1$ customers and Policy 2 at the k_i th customer results in a cost of $h_b \bar{\beta}(0 + 1 + \dots + k_i - 1) + h_a \bar{\alpha}(n - k_1 - \dots - k_i)$. For the case $i = p$, there is no cost for Policy 2 since all deliveries have been performed, hence the cost is $h_b \bar{\beta}(0 + 1 + \dots + k_p - 1)$. Defining $\mathbf{k} = (k_1, \dots, k_p)$, the optimization problem for this special case can be expressed as:

$$\begin{aligned} \text{(QP) minimize } g_1(\mathbf{k}) &= h_b \bar{\beta} \sum_{i=1, \dots, p} k_i(k_i - 1)/2 \\ &+ h_a \bar{\alpha} \left(n(p-1) - \sum_{i=1, \dots, p-1} (p-i)k_i \right) \end{aligned} \quad (3)$$

$$\text{subject to } \sum_{i=1, \dots, p} k_i = n \quad (4)$$

$$k_i \in \{1, \dots, n-p-1\}, \quad i \in \{1, \dots, p\}. \quad (5)$$

To further simplify the problem, we substitute (4) for k_p and relax (5) to inequality constraints to obtain:

$$\begin{aligned} \text{(RQP) minimize } g_2(\mathbf{k}) &= h_b \bar{\beta} \sum_{i=1, \dots, p-1} k_i(k_i - 1)/2 \\ &+ h_b \bar{\beta} \left(n - \sum_{i=1, \dots, p-1} k_i \right) \left(n - \sum_{i=1, \dots, p-1} k_i - 1 \right) / 2 \\ &+ h_a \bar{\alpha} \left(n(p-1) - \sum_{i=1, \dots, p-1} (p-i)k_i \right) \end{aligned} \quad (6)$$

$$\text{subject to } \sum_{i=1, \dots, p-1} k_i \leq n-1 \quad (7)$$

$$1 \leq k_i \leq n-p-1, \quad i \in \{1, \dots, p-1\}. \quad (8)$$

The gradient of $g_2(\mathbf{k})$ is then:

$$\nabla g_2(\mathbf{k}) = \left(\dots, h_b \bar{\beta} \left(k_i + \sum_{j=1, \dots, p-1} k_j - n \right) - h_a \bar{\alpha}(p-i), \dots \right). \quad (9)$$

The Hessian of $g_2(\mathbf{k})$ has entries equal to $2h_b \bar{\beta}$ on the main diagonal and $h_b \bar{\beta}$ otherwise, which yields a positive semi-definite matrix for positive h_b and $\bar{\beta}$. Setting the gradient equal to zero

results in

$$h_b \bar{\beta} \left(k_i + \sum_{j=1, \dots, p-1} k_j - n \right) = h_a \bar{\alpha}(p-i), \quad i \in \{1, \dots, p-1\}. \quad (10)$$

The equation system (10) can be further simplified to

$$k_i + \sum_{j=1, \dots, p-1} k_j = \frac{h_a \bar{\alpha}}{h_b \bar{\beta}} (p-i) + n, \quad i \in \{1, \dots, p-1\}. \quad (11)$$

Summing up Eq. (11) and dividing by p , we obtain

$$\sum_{i=1, \dots, p-1} k_i = \frac{h_a \bar{\alpha} p-1}{h_b \bar{\beta}} + n \frac{p-1}{p}. \quad (12)$$

Finally, substituting (12) into (11) and (4) yields

$$k_i^* = \frac{h_a \bar{\alpha}}{h_b \bar{\beta}} \left(\frac{p+1}{2} - i \right) + \frac{n}{p}, \quad i \in \{1, \dots, p\}. \quad (13)$$

Formula (13) gives the exact optimal solution to RQP, as long as constraints (7) and (8) are not violated. However, the value of p must still be determined. Note that the cost component related to the pickup commodities is a sum of quadratic terms, while the cost component associated with the delivery commodities is a sum of linear terms. Consequently, we want to maximize p in order to minimize the risk of incurring high quadratic costs. Using the observation that the k_i^* values are decreasing with respect to i and p , and the minimum of these values is k_p^* , we set $k_p^* = 1$ to maximize the value of p while respecting (7) and (8). Then, straightforward algebra yields

$$p^* = \left(1 - \frac{2h_b \bar{\beta}}{h_a \bar{\alpha}} + \sqrt{\left(\frac{2h_b \bar{\beta}}{h_a \bar{\alpha}} - 1 \right)^2 + 8n \frac{h_b \bar{\beta}}{h_a \bar{\alpha}}} \right) / 2. \quad (14)$$

We now focus on designing our linear time heuristic based on this analysis. We wish to decide when to apply Policy 2, while visiting the customers in the order in which they appear in the tour. Using (14) and (13) sequentially, we can compute the value of k_1^* in $O(1)$ time, which gives us an approximation for the first time Policy 2 should be used. The value of k_1^* can be further used to compute three other values at the first application of Policy 2: the amount of pickup commodities on board, the cost incurred due to pickup commodities, and the cost incurred due to delivery commodities. This yields a total of four thresholds which may trigger the application of Policy 2: (1) the number of customers visited; (2) the amount of pickup commodities on board; (3) the cost incurred due to pickup commodities; (4) the potential cost to be incurred due to delivery commodities.

The first three values increase with the number of visited customers, and the fourth one decreases. Note that the value of k_1^* (and the resulting thresholds) can be computed at any customer i by using $\bar{\alpha} = \sum_{j=i+1, \dots, n} \alpha_j / (n-i)$ and $\bar{\beta} = \sum_{j=i+1, \dots, n} \beta_j / (n-i)$. Using 10 000 randomly generated instances for each $|V_c| \in \{25, 50, 75, 100, 250, 500\}$, we have experimented with all 15 combinations of thresholds. The results are presented in Fig. 1 as box plots depicting the deviations from the optimal values. The best results are given by the most intuitive threshold, the amount of pickup commodities on board. The average deviation is 8.66% over 60 000 instances, whereas the maximal deviation is 52.70%. The values for the first quartile, the median, and the third quartile are 5.89%, 8.23%, and 10.86%, respectively. Fig. 2 also depicts the behavior of the best solution picked from all 15 alternatives for each instance, which behaves marginally better but requires 15 times more computational time. The behavior of threshold 2 with respect to the size of the instance is depicted in Fig. 2. Clearly, the maximum deviation of the method decreases monotonically and the average deviation decreases marginally as the instance size

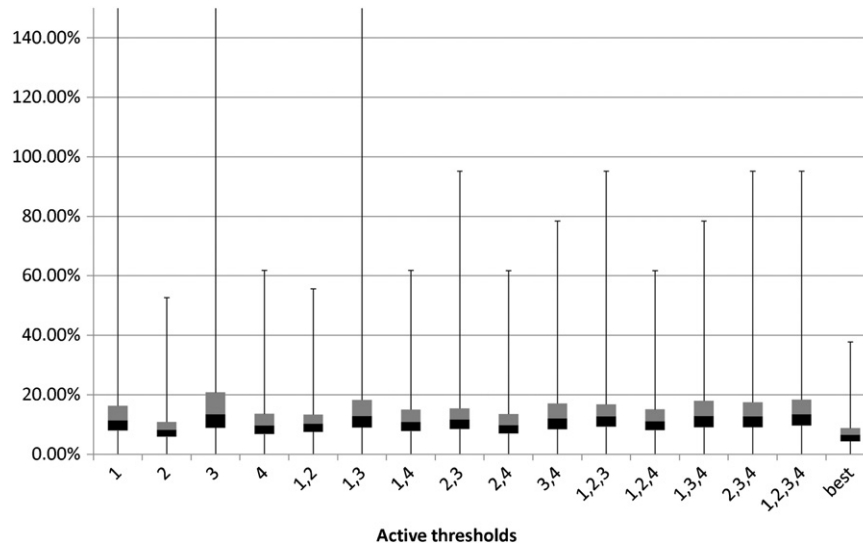


Fig. 1. Linear evaluation of the neighborhood: deviations obtained with different thresholds.

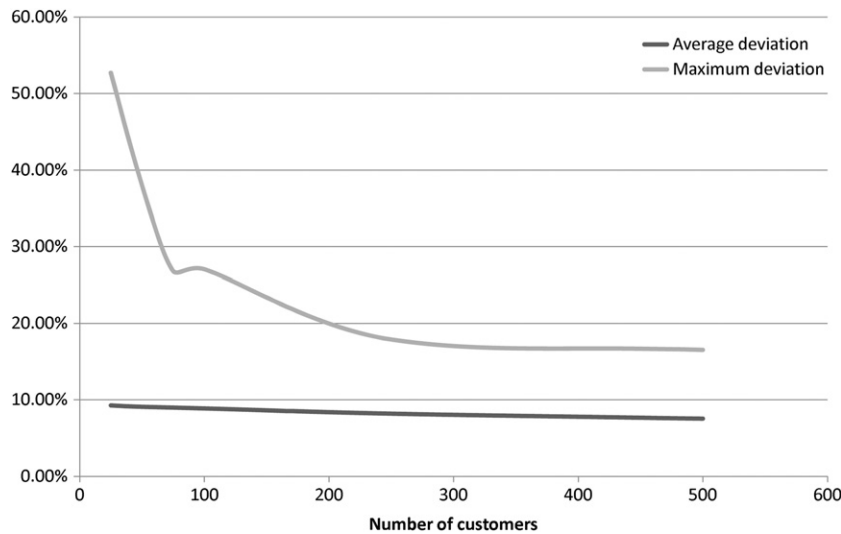


Fig. 2. Linear evaluation of the neighborhood: deviations by varying the problem size.

increases. These properties, combined with the linear time complexity, make this method a reasonable option for neighborhood evaluation.

3. Local search operators

Having analyzed how to evaluate moves in the neighborhood, we now turn our attention to the question of which neighborhoods are the most effective. We have analyzed classical neighborhoods from the routing literature, namely *relocate*, *2-opt*, and *swap*. The *relocate* neighborhood consists of the solutions obtained by relocating a single customer at a time. The number of possible relocations is $|V_c|(|V_c|-1)$. The *2-opt* neighborhood consists of solutions in which two non-consecutive edges are removed from the current solution and the resulting chains are reconnected differently. The number of possible moves in this neighborhood is $(|V_c|+1)(|V_c|-2)/2$. The *swap* neighborhood consists of all tours obtained by switching the positions of two customers in the tour. The number of swap moves in each neighborhood is $|V_c|(|V_c|-1)/2$. We did not examine richer neighborhoods, given the time complexity of the handling evaluation. Moreover, we have limited our

attention to moves that result in feasible tours to avoid introducing any parametric penalization and additional complexity during the search.

We have conducted preliminary experiments, in order to test the effectiveness of the local search operators. We have executed the *Tabu Search* (TS) algorithm (details are reported in Section 4), by considering instances with 20, 40, 60, 80, 100 customers (see Section 5, for the instance description) and the same initial tour. The TS algorithm was executed for 200 iterations by considering each neighborhood in turn, each possible pair of neighborhoods, and finally the combination of the three neighborhoods. At each iteration, the best non-tabu move was performed (with respect to the exact handling cost evaluation).

The average results obtained within five executions are reported in Fig. 3, where the percentage improvement with respect to the initial solution (i.e., *Imp*(%) in Fig. 3a) and the average computing time in seconds (i.e., *Time* (sec.) in Fig. 3b) are given for each configuration of the local search operators. The *2-opt* neighborhood is the most effective when applied alone. The average improvement is 2.85% and the average computing time is 14.15 s. The combination of *relocate* and *2-opt* produces an average improvement of 3.62% in 41.58 s, the three neighborhoods combined yield an improvement of 3.65% at the

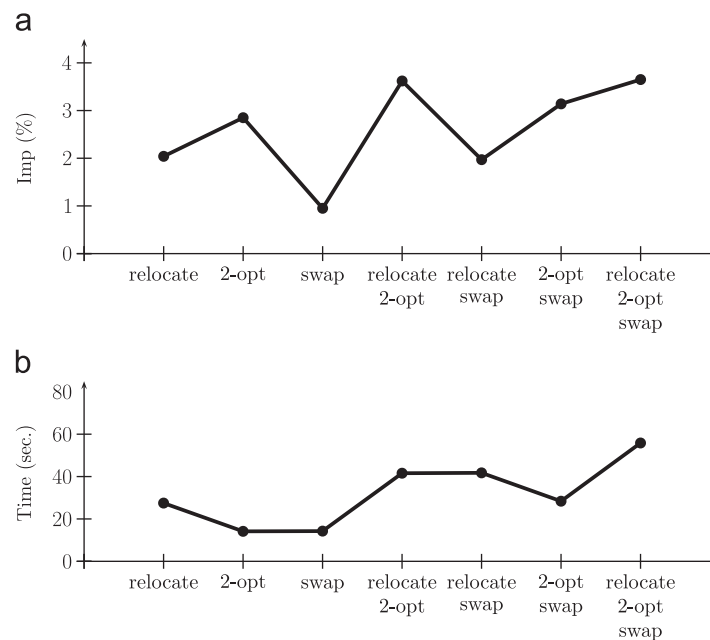


Fig. 3. Local search operators. (a) Average percentage improvement with different local search operators. (b) Average computing time with different local search operators.

expense of a yet another increase in the average computing time (55.84 s). Additional experiments were also conducted to assess the behavior of the three neighborhoods, when the heuristic evaluation of the moves is applied and when the algorithm is initialized with different tours. The performances are consistent with those just presented. Given these results, we decided to adopt the combination of relocate and 2-opt as neighborhood for our search.

4. Metaheuristics for the TSPPD-H

This paper presents the first heuristic attempts to solve the TSPPD-H. We have concentrated our efforts on the development, implementation and testing of classical local search-based metaheuristics. More precisely, we have developed a *Tabu Search* (TS) algorithm [16], an *Iterated Local Search* (ILS) algorithm [17], and an *Iterated Tabu Search* (ITS) algorithm [18]. Before describing each of these algorithms in detail, we should point out that we have tried to keep the algorithms simple by limiting the number of parameters involved and by making their implementation as natural and classical as possible. Moreover, we have imposed similar stopping conditions for the three algorithms, so as to perform fair comparisons.

There are three common aspects to the implementations of the algorithms. First, the same initial tour is provided to all three algorithms and is constructed through a two-phase approach as follows. A TSP tour through the depot and all customers is defined by executing the Lin–Kernighan (1973) algorithm, as implemented in the Concorde package, available online at <http://www.tsp.gatech.edu/concorde.html>. This algorithm is a fast and effective heuristic even for large-sized instances, and is also widely used. Note that no tuning has been performed on the Concorde implementation: the executable was used as available online. Given a TSP tour, we know from Mosheiov [19] that at least one feasible solution can be constructed for the SVPDP–P&D by relocating the depot along the tour. Among all feasible relocation positions, we choose the one for which the sum of routing and optimal handling costs is minimized. The time spent to determine the initial solution is negligible with respect to the execution of any of the metaheuristic approaches.

The second common aspect of the three algorithms is the effect of the initial solution. Our computational experience has

shown that it is also profitable to consider as a starting tour the reverse TSP tour. This tour has the same routing cost, but possibly a different handling cost. For this reason, we initialize our algorithms first with the original TSP tour found by the Lin–Kernighan algorithm, and then with the reverse one. The results in which the three algorithms are executed with the tour in a single direction, are denoted hereafter as TS-1dir, ILS-1dir, and ITS-1dir. The three algorithms in which two executions, one in each direction, are performed and the best solution reported, are denoted as TS-2dir, ILS-2dir, and ITS-2dir. In Section 5, the performance of each of the two approaches is reported.

Third, we found it relevant to test, in all algorithms, both the exact neighborhood evaluation and the heuristic neighborhood evaluation described in Section 2. However, once the best feasible move in the neighborhood is detected, we re-evaluate this move using the exact handling cost solver (i.e., the dynamic programming algorithm). In this way, we provide a more precise estimate of the move, without compromising the time complexity of the algorithms.

4.1. Tabu search algorithm

The TS pseudo-code is presented in Algorithm 4.1. The algorithm is initialized with the number of iterations to be performed, $N_{iter}=50, 100, 200$, and a feasible SVPDP–P&D tour, denoted as *Tour*. The cost of *Tour* is *costCurrent*. At each iteration of TS, all possible relocate and 2-opt moves are evaluated. The best feasible relocate move consisting of relocating the customer in position i to position p is memorized, as well as the best feasible 2-opt move, corresponding, say, to the positions i and j . The cost of the tour resulting from the best relocate move is denoted as *cost1*, and the cost of the tour resulting from the best 2-opt move is called *cost2*. Note that a move is the best feasible in the neighborhood either if its cost improves *costCurrent* (i.e., an *aspiration criterion* applied) or if it yields the lowest cost non-tabu solution in the neighborhood. If $cost1 < cost2$, the best feasible relocate move is implemented, otherwise the best 2-opt move is selected. In both cases, the tabu list is updated, the exact cost evaluation is performed (if the heuristic handling evaluation was previously adopted) and the algorithm proceeds to the next iteration.

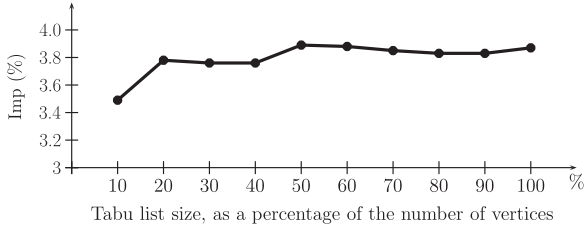


Fig. 4. Performance of TS, as a function of the tabu list size.

Note that, contrary to some implementations, the number of iterations is not reset to zero when an improving solution is detected. The rationale for this decision is to keep the computing time predictable and comparable with that of the other two algorithms.

Our tabu list stores pairs of locations. More precisely, if $(i, j), i \neq j$ is recorded in the tabu list, the associated swap operation on customers i and j is tabu and the associated 2-opt operation involving the i th arc and the j th arc in the tour is tabu. If an entry (i, i) is in the tabu list, the relocate operation involving customer i to any position is tabu. We have tested alternative tabu rules (e.g., any customer previously involved in a relocate or 2-opt move cannot be relocated or switched), but the one presented above proved to be the most effective.

The length of the tabu list was adjusted through an experimental analysis. We have executed TS by considering the 20, 40, 60, 80, and 100 vertex instances, and by linearly varying the length of the tabu list as a $c|V_c|$, where $c = 10\%, 20\%, \dots, 100\%$, with the exact evaluation of the handling cost. The results of this experiments are summarized in Fig. 4. The y-axis stands for the average improvement ($\text{Imp}(\%)$), with respect the initial solution. The best configuration is the one for which $c = 50\%$, where an average improvement of 3.89% is attained. This value proved to be robust in additional tests performed with different neighborhoods and initial solutions. Moreover, the c value proved to be consistent even for the ITS. We have also tried to randomly adjust the tabu list length, during the algorithm execution. The list length was randomly determined within a range proportional to V_c (e.g., $c \in [0\%, 0.50\%], c \in [25\%, 75\%]$). However, these experiments did not yield convincing results and we therefore decided to keep the tabu list length constant during the TS execution.

Algorithm 4.1. TS ($N_{\text{iter}}, \text{Tour}$)

```

costCurrent ← CostOfInitialTour
for i ← 1 to  $N_{\text{iter}}$ 
   $(i, p, \text{cost1}) \leftarrow \text{FINDBESTNOTTABU1OPTMOVE}(\text{Tour}, \text{costCurrent})$ 
   $(i, j, \text{cost2}) \leftarrow \text{FINDBESTNOTTABU2OPTMOVE}(\text{Tour}, \text{costCurrent})$ 
  if  $(\text{cost1} < \text{cost2})$ 
    IMPLEMENT1OPTMOVE( $\text{Tour}, i, p$ )
    UPDATETABULIST( $i, i$ )
  else
    IMPLEMENT2OPTMOVE( $\text{Tour}, i, j$ )
    UPDATETABULIST( $i, j$ )
  CostCurrent ← UPDATECOSTCURRENT()

```

4.2. Iterated local search

Algorithm 4.2. ILS ($N_{\text{iter}}, N_{\text{rand}}, \text{Tour}$)

```

costCurrent ← CostOfInitialTour
bestTour ← Tour
for it ← 1 to  $N_{\text{iter}}$ 
  Tour ← bestTour
  for r ← 1 to  $N_{\text{rand}}$ 
     $i, j \leftarrow \text{RandomNumber}(1, \dots, |V_c|)$ 

```

```

if  $(i = j)$ 
   $p \leftarrow \text{RandomNumber}(1, \dots, |V_c|, p \neq i)$ 
  1OPT( $\text{Tour}, i, p$ )
else
  2OPT( $\text{Tour}, i, j$ )
while (improvement)
  costNew ← PERFORM BEST MOVE( $\text{Tour}$ )
  if  $(\text{costNew} < \text{costCurrent})$ 
    costCurrent ← costNew
    bestTour ← Tour

```

The pseudo-code for ITS is provided in Algorithm 4.2. The randomized search in our ILS algorithm is performed by randomly selecting a pair of customer positions, $(i, j) \in V_c$. If $i = j$, an additional random value $p \neq i$ is selected and the customer in position i is relocated in position p . Otherwise, the chain of customers between positions i and j is reversed. The number of random moves to be performed is N_{rand} . Once the diversification step is concluded, the current solution is improved by our local search operators. While a feasible relocate or 2-opt move improves the current solution, the solution is updated and the search continues. If no such move exists, the iteration terminates. The next iteration is initialized with the *bestTour* found so far.

In order to define an appropriate N_{rand} value, we have executed the ILS for 200 iterations with instances up to 100 vertices, considering the exact evaluation of the neighborhood. We have set the number of random moves as $d|V_c|$, where $d = 5\%, 10\%, 15\%, 20\%, 25\%, 30\%, 40\%, 50\%, 75\%, 100\%$. The average percentage improvement $\text{Imp}(\%)$ achieved with respect to the initial solutions is reported in Fig. 5. A suitable value for d is 10%, producing an improvement of 1.29%.

4.3. Iterated tabu search

The ITS pseudo-code is provided in Algorithm 4.3. The first part is the same as in ILS, where the current solution is diversified by executing N_{rand} random moves. The diversified solution is then passed to the TS algorithm for improvements. The tabu list length in ILS is the same as in TS: $0.5|V_c|$. Similarly, N_{rand} is the same as in ILS: $0.1|V_c|$. The N_{iter}^* notation is used both for the number of executions of the outer ITS algorithm and for the number of iterations within each inner TS algorithm. We have scaled N_{iter}^* with respect to $N_{\text{iter}} = 50, 100, 200$ in TS and ILS, to obtain a fair comparison in terms of computing times and to verify whether it would be more convenient to explore the neighborhood in a deeper manner (i.e., as in TS) or diversify the search with a randomized approach (i.e., as in ITS). We have set $N_{\text{iter}}^* = \lfloor \sqrt{N_{\text{iter}}} \rfloor$, resulting in $N_{\text{iter}}^* = 7, 10, 14$. It can be observed in Section 5 that this choice guarantees comparable computing times with those of TS and ILS. Other schemes to set N_{iter}^* were experimented, but the one described above proved to be the most successful, both in terms of similarity of computing times and quality of the results attained by ITS.

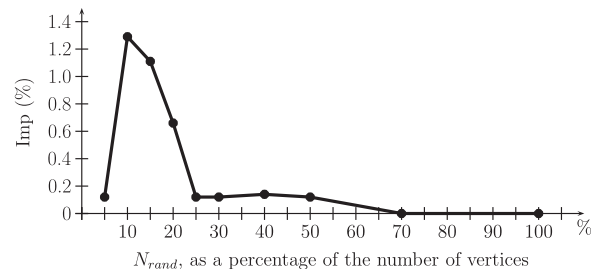


Fig. 5. Performance of ILS, as a function of N_{rand} .

Algorithm 4.3. ITS (N_{iter}^* , N_{rand} , $Tour$)

```

costCurrent ← CostOfInitialTour
bestTour ← Tour
for it ← 1 to  $N_{iter}^*$ 
    Tour ← bestTour
    for r ← 1 to  $N_{rand}$ 
        i, j ← RandomNumber(1, ...,  $|V_c|$ )
        if (i = j)
            p ← RandomNumber(1, ...,  $|V_c|$ ,  $p \neq i$ )
            1Opt(Tour, i, p)
        else 2Opt(Tour, i, j)
    costCurrent ← TABUSEARCH ( $N_{iter}^*$ , Tour)
    if (costNew < costCurrent)
        costCurrent ← costNew
        bestTour ← Tour

```

5. Computational results

Our algorithms were coded in C and run on an Intel Core 2 Quad, 2.83 GHz PC.

We have first tested our algorithms on the instances used in Battarra et al. [1]. The TS-2dir, ILS-2dir, and the ITS-2dir algorithms were executed for 200 iterations in each direction. We have excluded the instances with $|V_c| = 5$ from our analysis, given that the default Lin-Kernighan implementation in the Concorde package requires larger instances.

The number of instances for which TS-2dir, ILS-2dir, and the ITS-2dir algorithms produce worse solutions than the best known results reported in Battarra et al. [1] are 10, 8, and 8 out of 90, respectively. On the other hand, the number of instances for which the best known solutions are improved are 5, 3, and 5, respectively. The overall six new best known solution values are reported in Table 1. The instances are identified by the column labeled $Id. = 1, \dots, 10$. The column labeled h denotes value of the handling cost coefficients $h_a = h_b$. The best known solution values reported in Battarra et al. [1] are presented in the column labeled *Old best*, whereas the column for the updated best known results is denoted as *New best*.

The overall average deviation with respect to the best known solution values is 0.1% for the three algorithms, proving that small-size instances do not provide a clear answer about which algorithm is best for the TSPPD-H. However, the average deviation of the initial solution values (when both directions are considered) is 0.9%, proving that all algorithms were able to improve the initial solutions.

We have also tested our algorithms on a new benchmark set consisting of larger instances generated by following the same deterministic procedure as in Battarra et al. [1]. The 10 *Euclidean* instances for the SVPDP-P&D with 200 customers proposed by Gendreau et al. [20] (available online at the web-site <http://webpages.ull.es/users/hhperez/PDsite/index.html>) were selected and adapted to the TSPPD-H. For each instance, we have extracted smaller instances by considering the first 20, 40, 60, 80, 100, 120, 140, 160, 180, and 200 customers, respectively. The pickup and delivery values were also scaled as in Battarra et al. [1]. Denoting

the pickup value of customer $i = 1, \dots, |V_c|$ as p_i in the original SVPDP-P&D instance, we scaled it down to $p'_i = \max\{1, p_i \bmod 20\}$ and we then apportioned this quantity between α_i and β_i as

$$\beta_i = \lfloor p'_i(i \bmod 5)/5 \rfloor \quad \text{and} \quad \alpha_i = p'_i - \beta_i. \quad (15)$$

The vehicle capacity was set equal to $Q = \max\{\sum_{i \in V_c} \alpha_i, \sum_{i \in V_c} \beta_i\}$. In preliminary experiments we have analyzed the case $h_a = h_b = h$ and, we have observed that by keeping the product $h|V_c| = 20$ constant, we could construct instances having approximately equal routing and handling costs. Instances in which the handling cost dominates the routing cost may be solved by considering alternative solution shapes, and instances in which the routing cost is more important than the handling cost may be solved with a pure routing solver.

In Appendix A, we report the full results of our experiments. More precisely, we provide the initial TSPPD-H solution values in both directions, the minimum solution values attained in the current execution *Best*, and the results attained by TS-1dir, TS-2dir, ILS-1dir, ILS-2dir, ITS-1dir, and ITS-2dir. In what follows, we concentrate on aggregate results.

We first present a comparison of the performances of our three metaheuristics. In the following, we report the results of TS-1dir, ILS-1dir, and the ITS-1dir, when executed for 200 iterations using the exact neighborhood evaluation. Unfortunately, no lower bounds are available for the new instances, so we assess their performances by computing the deviations from the *Best* solution values. These results are summarized in Table 2, where column *Dev.(%)* denotes the percentage average deviation with respect to the best solutions, and column *Time (s)* denotes the average computing time in seconds. The last line reports the overall average results. The smallest deviations from the minimum over the three approaches are reported in boldface.

The TS-1dir algorithm proved to be the most effective, yielding an average deviation of 0.46%. The ITS-1dir produced a similar percentage average deviation of 0.83%. On the other hand, the ILS-1dir performs much worse than the other two methods, even if the average computing time is approximately doubled. Note that the ITS-1dir is better than TS-1dir when $|V_c| \in \{20, 40\}$. Since in such cases $N_{rand} = 2$ and 4, it seems that the number of random moves has to be very limited to derive a beneficial effect from diversification. In this study, the number of random moves increases linearly with V_c , slightly penalizing the ILS-1dir and ITS-1dir performance for large instances. However, we think it is fair to keep the implementation of the algorithms similar and we also avoid introducing sophisticated non-linear functions for defining the number of random moves. Consequently, we prefer the TS-1dir algorithm which performs well over all large set of instances without requiring too much parameter tuning. Finally, the average percentage deviation of the initial solutions with

Table 2

Comparison of the performances of TS-1dir, ILS-1dir, and ITS-1dir.

$ V_c $	TS-1dir		ILS-1dir		ITS-1dir	
	Dev. (%)	Time (s)	Dev. (%)	Time (s)	Dev. (%)	Time (s)
20	0.14	0.28	0.31	0.56	0.10	0.28
40	0.34	3.86	4.89	7.64	0.24	3.77
60	0.20	17.91	6.22	35.96	0.54	17.61
80	0.13	54.87	1.81	108.93	0.25	53.81
100	0.43	130.61	3.09	260.68	0.54	128.15
120	0.80	266.06	5.52	530.13	1.79	260.43
140	0.43	482.20	4.56	967.29	1.15	474.01
160	0.83	811.96	4.44	1628.12	1.26	798.29
180	0.61	1290.01	4.08	2582.38	0.86	1266.77
200	0.73	1948.39	3.59	3904.41	1.52	1914.78
Avg.	0.46	500.62	3.85	1002.61	0.83	491.79

Table 1

Old and new best known solution values.

$ V_c $	$Id.$	h	<i>Old best</i>	<i>New best</i>
20	2	0.5	532.5	531.5
20	5	0.5	536.5	531.5
20	7	0.5	544.5	542.5
20	8	0.5	510.0	509.0
15	5	1.0	551.0	546.0
15	10	1.0	455.0	454.0

respect to the best found is 4.54%. Both the TS-1dir algorithm and ITS-1dir produce remarkable improvements.

Given these results, we have analyzed the effect of initializing the algorithms with the TSP tour in a single direction and in two directions. The results are summarized in Table 3. The number of executions is kept to 200 and the handling cost is evaluated through the exact algorithm. The average percentage deviation with respect to the minimum result achieved is reported, when each algorithm (TS, ILS, and ITS) is executed in a single direction (1 dir.) or in both directions (2 dir.).

The TS-2dir algorithm improves the performance of TS-1dir considerably. The average percentage deviation decreases from 0.46% to 0.07%, the number of instances for which TS finds the best solution is 86 out of 100. Both ILS-2dir and ITS-2dir also improve the performance of their single direction counterpart, but the improvement is not as high. We do not report computing times, given that it is roughly twice what is required for a single direction (see Table 2). Even if the computing time is doubled, we consider this computing effort worthwhile.

In Table 4, the performance of TS, which proved to be the most consistent and effective algorithm, is tested by varying the number of iterations. The results obtained with the exact handling solver are reported. The columns 50, 100, and 200 iterations contain the average percentage deviations with respect to the *Best*, when TS-1dir and TS-2dir are executed for 50, 100, and 200 iterations, respectively. The last two lines of the table contain the overall percentage average deviation and the overall average computing time, in seconds.

The TS-2dir algorithm when executed for 50 iterations performs better than the TS-1dir with 100 iterations, and requires the same computing time. The same observation holds if TS-2dir is executed for 100 iterations and TS-1dir is executed for 200 iterations. These

results confirm that the TS-2dir algorithms outperforms TS-1dir. We have observed that it is better to perform half the number of iterations in each direction, rather than all of them in a single direction.

In the following, we analyze the performance of the heuristic neighborhood evaluation with respect to the exact algorithm. As discussed in Section 2, the strategy that proved to be the best to heuristically evaluate the neighborhood is the one based on the amount of pickup commodities on board. We apply this criterion whenever we evaluate the moves in the neighborhood, and we compare the performances of the algorithms when the exact and the heuristic evaluations are applied. We use the TS-2dir algorithm, which proved to be the most effective.

In Table 5, the results obtained by executing TS-2dir for 200 iterations with the *exact* and the *heuristic* evaluation of the neighborhood are reported. Each column reports the average percentage deviation from the *Best*, the last two lines report the overall average deviations and the average computing times, respectively. The difference in performance is quite impressive: the exact evaluation of the neighborhood produces results roughly 2.5% better than the ones obtained with the heuristic evaluation. It seems that an inexact evaluation of the neighborhood hinders the TS capability of escaping from local optima. On the other hand, the computing times are about 10 times smaller on average when the heuristic evaluation of the neighborhood is applied. Fig. 6 depicts the evolution of computing times as a function of instance size. The average computing time for the exact evaluation is shown with the dashed line and grows much faster than the average computing time of the heuristic neighborhood evaluation shown by the continuous line. We conclude that even if the exact neighborhood evaluation is much more effective, it is very time consuming for large size instances.

Finally, we should point out that our *Best* results depend strongly on the initial TSP solutions. We have tested the

Table 3
Comparison of the single and two-direction execution of TS, ILS, and ITS.

$ V_c $	TS		ILS		ITS	
	1 dir.	2 dir.	1 dir.	2 dir.	1 dir.	2 dir.
20	0.14	0.12	0.31	0.07	0.10	0.05
40	0.34	0.17	4.89	3.44	0.24	0.10
60	0.20	0.07	6.22	2.61	0.54	0.13
80	0.13	0.13	1.81	1.08	0.25	0.05
100	0.43	0.19	3.09	1.08	0.54	0.34
120	0.80	0.00	5.52	4.35	1.79	1.32
140	0.43	0.00	4.56	3.53	1.15	1.03
160	0.83	0.00	4.44	3.85	1.26	1.25
180	0.61	0.00	4.08	3.28	0.86	0.86
200	0.73	0.00	3.59	3.51	1.52	1.52
Avg.	0.46	0.07	3.85	2.68	0.83	0.66

Table 4
Performance of TS, by varying the number of iterations.

$ V_c $	50 iter.		100 iter.		200 iter.	
	1 dir.	2 dir.	1 dir.	2 dir.	1 dir.	2 dir.
20	0.29	0.12	0.14	0.12	0.14	0.12
40	0.42	0.19	0.35	0.18	0.34	0.17
60	0.64	0.29	0.54	0.23	0.20	0.07
80	0.24	0.20	0.24	0.20	0.13	0.13
100	0.54	0.29	0.43	0.19	0.43	0.19
120	1.20	0.26	1.17	0.23	0.80	0.00
140	0.59	0.10	0.53	0.08	0.43	0.00
160	1.11	0.34	1.04	0.11	0.83	0.00
180	0.70	0.22	0.61	0.12	0.61	0.00
200	1.06	0.18	0.77	0.02	0.73	0.00
Avg.	0.68	0.22	0.58	0.15	0.46	0.07
Time (s)	125.26	250.36	250.44	500.57	500.62	1000.69

Table 5
Comparative performance of TS-2dir, with an exact and a heuristic evaluation of the neighborhood.

$ V_c $	Exact evaluation, 200 iter.	Heuristic evaluation, 200 iter.
20	0.12	1.35
40	0.17	2.49
60	0.07	2.22
80	0.13	0.77
100	0.19	0.96
120	0.00	4.14
140	0.00	3.39
160	0.00	3.81
180	0.00	3.14
200	0.00	3.50
Avg.	0.07	2.58
Time (s)	1000.69	117.22

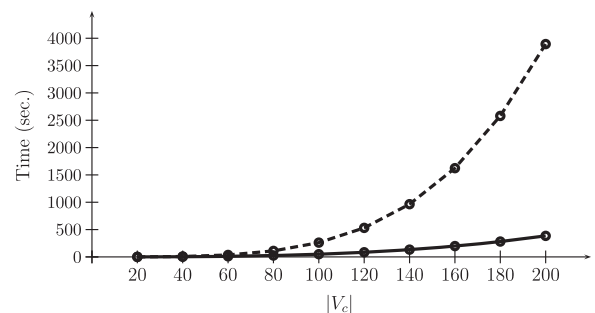


Fig. 6. Computing time, as a function of $|V_c|$, and under the heuristic or the exact neighborhood evaluation.

robustness of our results by executing five additional experiments to verify the behavior of the algorithms with different initial solutions (the Lin–Kerningham does not return the same TSP tour at each execution). The deviations of our results with respect to the *Best* is only -0.04% , 0.00% , -0.03% , -0.04% , -0.05% , proving that the results are consistent on average. However, the maximal deviation and the standard deviation are 2.65% , 2.00% , -2.47% , 2.74% , -2.07% and 0.55% , 0.57% , 0.52% , 0.66% , 0.56% , because the results are affected by the initial solutions.

6. Conclusions

We have designed, implemented and tested three metaheuristics for the TSPPD-H which includes both routing and handling costs in its objective function. We have first analyzed the handling cost subproblem in detail. We have then constructed an exact $O(n^2)$ dynamic programming algorithm and an approximate $O(n)$ heuristic algorithm, the latter based on the closed form solution of a special case of the subproblem. Using these heuristics, we have devised three metaheuristic algorithms, a TS, an ILS, and an ITS. We have extensively tested our algorithms on instances adapted from the TSPPD literature. Our results show that the TS algorithm using the exact dynamic programming algorithm yields the best results, and replacing the exact algorithm with the

approximate one results in a considerably faster algorithm at the cost of slightly worse solutions.

Acknowledgements

This work was partly funded by the Canadian Natural Science and Engineering Research Council under Grant 39682-10 and by Ministero dell'Istruzione, dell'Università e della Ricerca, Italy. This support is gratefully acknowledged. Thanks are due to the referees for their valuable comments.

Appendix A. Detailed computational results

In the following, we report the solution values we have referred in Section 5. Tables 6 and 7 display the results obtained by the TS-1dir and TS-2dir algorithms, when executed for 50, 100, and 200 iterations and by applying both the *exact* and *heuristic* evaluation of the neighborhood. Note that the columns denoted by *Initial Solution* report the initial TSPPD-H solution cost for both tour directions. The column *Best* reports the minimum cost solution detected with all the algorithms, when initialized with these initial solutions. The last line of the tables report the average overall computing times. Tables 8 and 9 report the same

Table 6
Detailed results for TS with heuristic neighborhood evaluation (first part).

$ V_c $	Id.	Best	Initial solution		TS heuristic						TS exact							
			1 dir.		2 dir.		1 dir.			2 dir.			1 dir.			2 dir.		
							50 iter.	100 iter.	200 iter.	50 iter.	100 iter.	200 iter.	50 iter.	100 iter.	200 iter.	50 iter.	100 iter.	200 iter.
20	1	633.00	634.00	743.00	634.00	634.00	634.00	634.00	634.00	634.00	634.00	633.00	633.00	633.00	633.00	633.00	633.00	
	2	584.00	587.00	621.00	587.00	587.00	587.00	590.00	590.00	590.00	584.00	584.00	584.00	589.00	589.00	589.00		
	3	573.00	590.00	598.00	590.00	590.00	590.00	590.00	598.00	598.00	573.00	573.00	573.00	573.00	573.00	573.00		
	4	706.00	773.00	728.00	718.00	718.00	718.00	712.00	712.00	712.00	706.00	706.00	706.00	707.00	706.00	706.00		
	5	501.00	532.00	563.00	507.00	507.00	507.00	510.00	510.00	510.00	501.00	501.00	501.00	503.00	501.00	501.00		
	6	578.00	661.00	578.00	578.00	578.00	578.00	578.00	578.00	578.00	578.00	578.00	578.00	578.00	578.00	578.00		
	7	612.00	619.00	685.00	619.00	619.00	619.00	667.00	667.00	667.00	612.00	612.00	612.00	612.00	612.00	612.00		
	8	567.00	628.00	580.00	577.00	577.00	577.00	571.00	571.00	571.00	569.00	569.00	569.00	569.00	569.00	569.00		
	9	604.00	697.00	629.00	625.00	625.00	625.00	625.00	625.00	625.00	616.00	607.00	607.00	606.00	606.00	606.00		
	10	574.00	616.00	588.00	595.00	595.00	595.00	588.00	588.00	588.00	577.00	577.00	577.00	586.00	586.00	586.00		
40	1	909.50	915.50	918.50	915.50	915.50	915.50	918.50	918.50	918.50	911.00	909.50	909.50	918.50	918.50	918.50		
	2	885.00	909.00	904.00	909.00	907.00	905.00	902.00	902.00	902.00	898.00	898.00	898.00	887.50	887.50	887.50		
	3	815.50	889.50	876.50	882.50	882.50	882.50	864.00	864.00	864.00	824.00	820.00	820.00	815.50	815.50	815.50		
	4	898.00	936.00	968.50	919.50	919.50	919.50	960.50	960.50	960.50	898.00	898.00	898.00	898.00	898.00	898.00		
	5	743.50	800.50	751.50	791.00	791.00	791.00	751.50	751.50	751.50	744.50	744.50	744.50	745.50	745.50	745.50		
	6	901.00	935.50	903.00	935.50	935.50	935.50	903.00	903.00	903.00	901.00	901.00	901.00	901.00	901.00	901.00		
	7	798.50	889.00	865.00	853.00	850.50	835.00	833.50	833.50	833.50	804.00	804.00	804.00	852.00	804.00	804.00		
	8	795.00	830.50	828.00	811.00	811.00	811.00	828.00	828.00	828.00	797.50	797.50	797.50	812.50	812.50	812.50		
	9	876.50	895.50	998.50	895.50	895.50	895.50	991.00	991.00	981.00	876.50	876.50	876.50	876.50	876.50	876.50		
	10	866.00	918.00	901.50	918.00	918.00	918.00	901.50	901.50	901.50	869.00	869.00	868.00	870.50	870.50	868.00		
60	1	1060.12	1063.79	1138.54	1063.79	1063.79	1063.79	1126.35	1126.35	1126.35	1061.11	1061.11	1060.13	1096.87	1096.87	1096.87		
	2	1051.13	1088.69	1060.91	1086.96	1086.96	1086.96	1060.91	1060.91	1060.91	1051.13	1051.13	1051.13	1051.13	1051.13	1051.13		
	3	990.36	1012.39	1098.76	1012.39	1012.39	1012.39	1041.05	1041.05	1041.05	997.18	997.18	997.18	997.18	997.18	997.18		
	4	1061.45	1164.26	1086.32	1114.14	1114.14	1114.14	1086.32	1086.32	1086.32	1061.45	1061.45	1061.45	1061.45	1061.45	1061.45		
	5	986.93	1068.87	1073.52	1033.69	1033.69	1033.69	1041.86	1041.86	1041.86	986.93	986.93	986.93	1003.04	1003.04	1003.04		
	6	1086.31	1131.28	1132.81	1131.28	1131.28	1131.28	1132.81	1132.81	1132.81	1106.17	1099.50	1086.31	1106.93	1099.59	1099.59		
	7	1005.43	1097.43	1020.94	1097.43	1097.43	1097.43	1017.97	1017.97	1017.97	1040.43	1036.75	1018.95	1005.43	1005.43	1005.43		
	8	1027.19	1127.12	1059.54	1127.12	1127.12	1127.12	1059.54	1059.54	1059.54	1027.19	1027.19	1027.19	1045.24	1027.19	1027.19		
	9	1001.43	1133.31	1013.90	1079.15	1079.15	1079.15	1013.90	1013.90	1013.90	1002.15	1002.15	1001.43	1002.15	1002.15	1001.43		
	10	1062.11	1081.74	1146.98	1081.74	1081.74	1081.74	1142.98	1142.98	1142.98	1064.83	1064.83	1062.11	1064.83	1064.83	1062.11		
80	1	951.63	971.25	1008.88	961.75	961.75	961.75	978.50	978.50	978.50	951.63	951.63	951.63	951.63	951.63	951.63		
	2	960.38	989.63	973.88	989.63	989.63	989.63	973.88	973.88	973.88	973.13	973.13	962.75	969.00	969.00	962.88		
	3	958.88	1027.75	973.50	978.25	978.25	978.25	969.63	969.63	969.63	959.13	959.13	959.13	959.13	959.13	959.13		
	4	1009.38	1017.50	1059.00	1017.50	1017.50	1017.50	1030.50	1030.50	1030.50	1009.38	1009.38	1009.38	1009.38	1009.38	1009.38		
	5	999.38	1002.88	1019.50	1002.88	1002.88	1002.88	1019.50	1019.50	1019.50	999.38	999.38	999.38	1011.00	1008.25	1002.88		
	6	1032.00	1047.13	1065.00	1047.13	1047.13	1047.13	1065.00	1065.00	1065.00	1042.75	1042.75	1042.63	1042.75	1042.75	1042.63		

Table 6 (continued)

$ V_c $	Id.	Best	Initial solution		TS heuristic						TS exact					
			1 dir.	2 dir.	1 dir.			2 dir.			1 dir.			2 dir.		
					50 iter.	100 iter.	200 iter.	50 iter.	100 iter.	200 iter.	50 iter.	100 iter.	200 iter.	50 iter.	100 iter.	200 iter.
	7	935.25	943.88	960.38	943.88	943.88	943.88	960.38	960.38	960.38	935.25	935.25	935.25	935.25	935.25	935.25
	8	982.13	1000.00	1007.38	1000.00	1000.00	1000.00	982.88	982.88	982.88	982.13	982.13	982.13	982.13	982.13	982.13
	9	952.13	954.38	1062.50	954.38	954.38	954.38	960.75	960.75	960.75	952.13	952.13	952.13	956.38	953.50	953.50
	10	960.50	962.88	1000.25	962.88	962.88	962.88	989.13	989.13	989.13	960.50	960.50	960.50	977.13	977.13	977.13
100	1	1046.20	1068.10	1084.80	1068.10	1068.10	1068.10	1078.50	1078.50	1078.50	1067.50	1067.50	1067.50	1046.20	1046.20	1046.20
	2	1090.70	1095.30	1137.60	1095.30	1095.30	1095.30	1117.20	1117.20	1117.20	1094.50	1094.50	1094.50	1090.70	1090.70	1090.70
	3	1110.30	1125.40	1158.70	1122.00	1122.00	1122.00	1153.90	1153.90	1153.90	1113.20	1113.20	1113.20	1113.20	1113.20	1113.20
	4	1116.80	1153.10	1144.40	1153.10	1153.10	1153.10	1136.20	1136.20	1136.20	1128.20	1116.80	1116.80	1128.20	1128.20	1128.20
	5	1073.00	1100.00	1097.80	1100.00	1100.00	1100.00	1097.80	1097.80	1097.80	1089.70	1089.70	1089.70	1093.00	1093.00	1093.00
	6	1147.40	1174.20	1148.00	1174.20	1174.20	1174.20	1148.00	1148.00	1148.00	1147.40	1147.40	1147.40	1147.40	1147.40	1147.40
	7	1076.80	1113.00	1078.60	1110.70	1110.70	1110.70	1078.60	1078.60	1078.60	1076.80	1076.80	1076.80	1076.80	1076.80	1076.80
	8	1131.70	1177.10	1143.40	1140.80	1140.80	1140.80	1143.40	1143.40	1143.40	1131.70	1131.70	1131.70	1131.70	1131.70	1131.70
	9	1081.40	1159.40	1081.80	1122.60	1122.60	1122.60	1081.80	1081.80	1081.80	1082.50	1081.50	1081.50	1081.40	1081.40	1081.40
	10	1123.80	1172.50	1133.70	1153.70	1153.70	1153.70	1133.70	1133.70	1133.70	1124.50	1124.50	1124.50	1124.50	1124.50	1124.50

Table 7

Detailed results for TS with heuristic neighborhood evaluation (second part).

$ V_c $	Id.	Best	Initial solution		TS heuristic						TS exact					
			1 dir.	2 dir.	1 dir.			2 dir.			1 dir.			2 dir.		
					50 iter.	100 iter.	200 iter.	50 iter.	100 iter.	200 iter.	50 iter.	100 iter.	200 iter.	50 iter.	100 iter.	200 iter.
120	1	1472.49	1575.08	1526.53	1575.08	1575.08	1575.08	1526.53	1526.53	1526.53	1489.13	1489.13	1475.43	1472.49	1472.49	1472.49
	2	1482.29	1540.24	1593.14	1540.24	1540.24	1540.24	1564.68	1564.68	1564.68	1510.17	1510.17	1510.17	1482.29	1482.29	1482.29
	3	1510.05	1550.10	1676.84	1550.10	1550.10	1550.10	1676.84	1676.84	1676.84	1515.03	1510.05	1510.05	1516.88	1516.88	1516.88
	4	1563.50	1664.65	1661.45	1664.65	1664.65	1664.65	1628.78	1628.78	1628.78	1579.75	1579.75	1572.14	1573.55	1573.55	1563.50
	5	1457.05	1542.96	1599.23	1542.96	1542.96	1542.96	1599.23	1599.23	1599.23	1476.65	1476.65	1470.28	1462.27	1462.27	1457.05
	6	1546.25	1622.27	1670.92	1622.27	1622.27	1622.27	1670.92	1670.92	1670.92	1593.23	1593.23	1593.23	1546.25	1546.25	1546.25
	7	1557.27	1648.08	1610.88	1648.08	1648.08	1648.08	1610.88	1610.88	1610.88	1565.92	1565.92	1565.92	1563.82	1563.82	1557.27
	8	1524.07	1620.30	1599.61	1620.30	1620.30	1620.30	1599.61	1599.61	1599.61	1524.07	1524.07	1524.07	1558.35	1558.35	1548.45
	9	1547.95	1645.21	1620.27	1645.21	1645.21	1645.21	1620.27	1620.27	1620.27	1561.89	1561.89	1547.95	1561.20	1561.20	1552.60
	10	1573.87	1667.93	1622.36	1667.93	1667.93	1667.93	1622.36	1622.36	1622.36	1602.53	1602.53	1588.19	1573.87	1573.87	1573.87
140	1	1575.20	1654.40	1668.04	1654.40	1654.40	1654.40	1645.60	1645.60	1645.60	1588.74	1588.74	1588.74	1575.20	1575.20	1575.20
	2	1605.80	1670.76	1709.20	1670.76	1670.76	1670.76	1706.56	1706.56	1706.56	1635.54	1626.22	1626.22	1614.94	1614.94	1605.80
	3	1583.68	1648.92	1751.46	1634.40	1634.40	1634.40	1751.46	1751.46	1751.46	1583.68	1583.68	1583.68	1654.10	1627.52	1615.02
	4	1712.66	1787.94	1760.56	1774.64	1774.64	1774.64	1760.56	1760.56	1760.56	1731.70	1731.70	1731.70	1712.66	1712.66	1712.66
	5	1547.40	1633.14	1610.50	1633.14	1633.14	1633.14	1610.50	1610.50	1610.50	1572.40	1572.40	1564.46	1547.40	1547.40	1547.40
	6	1662.28	1759.38	1741.18	1759.38	1759.38	1759.38	1741.18	1741.18	1741.18	1662.28	1662.28	1662.28	1691.26	1691.26	1688.36
	7	1664.42	1749.36	1691.36	1719.60	1719.60	1719.60	1691.36	1691.36	1691.36	1664.42	1664.42	1664.42	1664.42	1664.42	1664.42
	8	1646.08	1741.96	1698.12	1705.24	1705.24	1705.24	1698.12	1698.12	1698.12	1646.08	1646.08	1646.08	1650.00	1650.00	1650.00
	9	1629.10	1700.02	1730.10	1700.02	1700.02	1700.02	1730.10	1730.10	1730.10	1636.02	1636.02	1629.10	1636.64	1632.18	1632.18
	10	1693.82	1717.34	1777.14	1717.34	1717.34	1717.34	1777.14	1777.14	1777.14	1693.82	1693.82	1693.82	1693.82	1693.82	1693.82
160	1	1741.75	1845.00	1836.63	1845.00	1845.00	1845.00	1836.63	1836.63	1836.63	1756.13	1756.13	1756.13	1748.88	1748.88	1741.75
	2	1773.63	1820.50	1856.75	1820.50	1820.50	1820.50	1856.75	1856.75	1856.75	1791.88	1791.88	1791.88	1773.63	1773.63	1773.63
	3	1690.88	1748.38	1828.25	1748.38	1748.38	1748.38	1757.13	1757.13	1757.13	1738.38	1726.88	1726.88	1702.13	1690.88	1690.88
	4	1858.13	1964.75	1962.63	1964.75	1964.75	1964.75	1962.63	1962.63	1962.63	1858.13	1858.13	1858.13	1865.88	1865.88	1865.88
	5	1667.75	1765.13	1756.75	1765.13	1765.13	1765.13	1756.75	1756.75	1756.75	1711.88	1711.88	1703.25	1673.38	1673.38	1667.75
	6	1813.00	1865.88	1844.50	1865.88	1865.88	1865.88	1844.50	1844.50	1844.50	1824.25	1824.25	1824.25	1813.00	1813.00	1813.00
	7	1774.25	1835.38	1871.00	1835.38	1835.38	1835.38	1871.00	1871.00	1871.00	1811.00	1811.00	1789.13	1802.38	1774.25	1774.25
	8	1770.75	1825.25	1852.88	1825.25	1825.25	1825.25	1820.50	1820.50	1820.50	1779.13	1779.13	1779.13	1770.75	1770.75	1770.75
	9	1800.88	1909.50	1895.25	1893.50	1893.50	1893.50	1895.25	1895.25	1895.25	1807.25	1807.25	1800.88	1826.88	1826.88	1809.75
	10	1764.38	1860.13	1809.38	1860.13	1860.13	1860.13	1809.38	1809.38	1809.38	1768.50	1768.50	1768.50	1764.38	1764.38	1764.38
180	1	1854.21	1936.01	1937.75	1936.01	1936.01	1936.01	1937.75	1937.75	1937.75	1862.53	1854.21	1854.21	1860.88	1860.88	1860.88
	2	1863.32	1959.69	1927.48	1959.69	1959.69	1959.69	1927.48	1927.48	1927.48	1880.17	1880.17	1880.17	1872.05	1872.05	1863.32
	3	1858.41	1947.95	1889.87	1926.52	1926.52	1889.87	1889.87	1889.87	1889.87	1873.42	1873.42	1873.42	1858.41	1858.41	1858.41
	4	1988.19	2067.97	2059.51	2067.97	2067.97	2059.51	2059.51	2059.51	2059.51	2018.57	2010.19	2009.80	2013.75	2001.16	1988.19
	5	1795.81	1869.75	1897.19	1869.75	1869.75	1869.75	1853.72	1853.72	1853.72	1797.06	1797.06	1797.06	1811.58	1811.58	1795.81
	6	1817.47	1873.17	1938.34	1873.17	1873.17	1938.34	1938.34	1938.34	1938.34	1845.49	1845.49	1845.49	1817.47	1817.47	1817.47
	7	1868.04	1929.56	1948.30	1929.56	1929.56	1948.30	1948.30	1948.30	1948.30	1868.04	1868.04	1868.04	1887.59	1887.59	1887.59
	8	1883.40	1964.13	1940.26	1930.46	1930.46	1930.46	1940.26	1940.26	1940.26	1895.90	1895.90	1895.90	1883.51	1883.40	1883.40
	9	1931.44	2004.60	2018.87	2004.60	2004.60	2004.60	2018.87	2018.87	2018.87	1931.44	1931.44	1931.44	1956.91	1956.91	1956.91
	10	1852.51	1923.11	1896.85	1923.11	1923.11	1923.11	1896.85	1896.85	1896.85	1871.01	1871.01	1871.01	1852.51	1852.51	1852.51
200	1	1976.30	2060.60	2065.70	2060.60	2060.60	2060.60	2065.70	2065.70	2065.70	2031.90	2031.90	2031.90	1976.30	1976.30	1976.30
	2	1982.00	2074.90	2077.60	2074.90	2074.90	2074.90	2077.60	2077.60	2077.60	1995.00	1995.00	1987.40	1982.00	1982.00	1982.00
	3	1976.70	2037.10	2090.90	2037.10	2037.10	2037.10	2054.90	2054.90	2054.90	1991.10	1991.10	1991.10	1976.70	1976.70	1976.70

Table 7 (continued)

$ V_c $	Id.	Best	Initial solution		TS heuristic						TS exact					
			1 dir.	2 dir.	1 dir.			2 dir.			1 dir.			2 dir.		
					50 iter.	100 iter.	200 iter.	50 iter.	100 iter.	200 iter.	50 iter.	100 iter.	200 iter.	50 iter.	100 iter.	200 iter.
	4	2119.50	2220.30	2211.90	2220.30	2220.30	2220.30	2211.90	2211.90	2211.90	2182.10	2182.10	2182.10	2119.50	2119.50	2119.50
	5	1905.60	1974.20	1986.80	1974.20	1974.20	1974.20	1986.80	1986.80	1986.80	1914.80	1914.80	1914.80	1918.50	1909.00	1905.60
	6	2011.10	2042.30	2088.00	2042.30	2042.30	2042.30	2088.00	2088.00	2088.00	2012.00	2012.00	2012.00	2011.10	2011.10	2011.10
	7	1983.00	2049.20	2048.90	2049.20	2049.20	2049.20	2046.50	2046.50	2046.50	1983.00	1983.00	1983.00	2009.10	2009.10	2009.10
	8	2000.30	2104.00	2096.30	2104.00	2104.00	2104.00	2096.30	2096.30	2096.30	2048.60	2000.30	2000.30	2016.20	2016.20	2016.20
	9	2052.70	2131.60	2141.10	2131.60	2131.60	2131.60	2141.10	2141.10	2141.10	2052.70	2052.70	2052.70	2069.70	2069.70	2069.70
	10	1977.90	2009.40	2066.30	2009.40	2009.40	2009.40	2066.30	2066.30	2066.30	1988.30	1977.90	1977.90	1995.90	1995.90	1990.40
Time (s)					14.65103	29.30	58.61	14.65	29.30	58.61	125.26	250.44	500.62	125.10	250.13	500.07

Table 8

Detailed results for ILS and ITS with 200 iterations (first part).

$ V_c $	Id.	Best	Initial solution		ILS heuristic		ILS exact		ITS heuristic		ITS exact	
			1 dir.	2 dir.	1 dir. 200 iter.	2 dir. 200 iter.	1 dir. 200 iter.	2 dir. 200 iter.	1 dir. 200 iter.	2 dir. 200 iter.	1 dir. 200 iter.	2 dir. 200 iter.
20	1	633.00	634.00	743.00	634.00	634.00	633.00	634.00	634.00	652.00	634.00	633.00
	2	584.00	587.00	621.00	587.00	590.00	584.00	610.00	587.00	589.00	584.00	584.00
	3	573.00	590.00	598.00	588.00	576.00	587.00	573.00	590.00	594.00	573.00	573.00
	4	706.00	773.00	728.00	713.00	720.00	706.00	706.00	718.00	712.00	706.00	706.00
	5	501.00	532.00	563.00	501.00	506.00	501.00	514.00	507.00	512.00	501.00	501.00
	6	578.00	661.00	578.00	578.00	578.00	578.00	578.00	578.00	578.00	578.00	578.00
	7	612.00	619.00	685.00	615.00	659.00	612.00	619.00	619.00	660.00	612.00	626.00
	8	567.00	628.00	580.00	575.00	567.00	567.00	567.00	577.00	569.00	569.00	567.00
	9	604.00	697.00	629.00	616.00	614.00	608.00	612.00	623.00	623.00	604.00	607.00
	10	574.00	616.00	588.00	592.00	586.00	574.00	586.00	579.00	588.00	577.00	577.00
40	1	909.50	915.50	918.50	915.50	918.50	915.50	918.50	915.50	918.50	911.00	909.50
	2	885.00	909.00	904.00	909.00	904.00	909.00	904.00	909.00	888.00	891.00	885.00
	3	815.50	889.50	876.50	889.50	876.50	889.50	876.50	831.50	834.50	825.50	823.50
	4	898.00	936.00	968.50	936.00	968.50	936.00	968.50	919.50	917.00	898.00	898.00
	5	743.50	800.50	751.50	800.50	751.50	800.50	751.50	766.00	751.50	743.50	743.50
	6	901.00	935.50	903.00	935.50	903.00	935.50	903.00	935.50	903.00	901.00	901.00
	7	798.50	889.00	865.00	889.00	865.00	879.00	865.00	853.00	834.50	798.50	798.50
	8	795.00	830.50	828.00	830.50	828.00	830.50	828.00	808.50	819.50	795.00	795.00
	9	876.50	895.50	998.50	895.50	998.50	895.50	998.50	895.50	990.00	876.50	876.50
	10	866.00	918.00	901.50	918.00	901.50	900.50	901.50	918.00	901.50	869.00	866.00
60	1	1060.12	1063.79	1138.54	1063.79	1138.54	1063.79	1138.54	1063.79	1138.54	1061.11	1060.12
	2	1051.13	1088.69	1060.91	1088.69	1060.91	1088.69	1060.91	1086.96	1060.91	1054.64	1051.13
	3	990.36	1012.39	1098.76	1012.39	1098.76	1012.39	1098.76	1012.39	1098.76	990.36	998.15
	4	1061.45	1164.26	1086.32	1164.26	1086.32	1164.26	1086.32	1114.14	1086.32	1061.45	1061.45
	5	986.93	1068.87	1073.52	1068.87	1073.52	1068.87	1073.52	1033.69	1041.86	986.93	986.93
	6	1086.31	1131.28	1132.81	1131.28	1132.81	1131.28	1132.81	1131.28	1132.81	1095.00	1090.50
	7	1005.43	1097.43	1020.94	1097.43	1020.94	1097.43	1020.94	1097.43	1020.94	1040.43	1012.84
	8	1027.19	1127.12	1059.54	1127.12	1059.54	1127.12	1059.54	1117.40	1059.54	1027.19	1031.33
	9	1001.43	1133.31	1013.90	1133.31	1013.90	1133.31	1013.90	1061.80	1013.90	1002.09	1004.54
	10	1062.11	1081.74	1146.98	1081.74	1146.98	1081.74	1146.98	1081.74	1146.98	1068.93	1062.91
80	1	951.63	971.25	1008.88	971.25	1008.88	971.25	1008.88	961.75	1001.88	956.50	969.38
	2	960.38	989.63	973.88	989.63	973.88	989.63	973.88	989.63	973.88	969.13	960.38
	3	958.88	1027.75	973.50	1027.75	973.50	1027.75	973.50	978.25	973.50	958.88	959.13
	4	1009.38	1017.50	1059.00	1017.50	1059.00	1017.50	1059.00	1015.38	1039.63	1009.38	1009.38
	5	999.38	1002.88	1019.50	1002.88	1019.50	1002.88	1019.50	1002.88	1019.50	999.38	999.38
	6	1032.00	1047.13	1065.00	1047.13	1065.00	1047.13	1065.00	1047.13	1065.00	1042.75	1032.00
	7	935.25	943.88	960.38	943.88	960.38	943.88	960.38	943.88	960.38	935.25	935.25
	8	982.13	1000.00	1007.38	1000.00	1007.38	1000.00	1007.38	1000.00	1007.38	982.13	987.13
	9	952.13	954.38	1062.50	954.38	1062.50	954.38	1062.50	954.38	960.88	952.13	952.13
	10	960.50	962.88	1000.25	962.88	1000.25	962.88	1000.25	962.88	991.38	960.50	960.50
100	1	1046.20	1068.10	1084.80	1068.10	1084.80	1068.10	1084.80	1068.10	1084.80	1067.50	1068.40
	2	1090.70	1095.30	1137.60	1095.30	1137.60	1095.30	1137.60	1095.30	1137.60	1094.50	1094.10
	3	1110.30	1125.40	1158.70	1125.40	1158.70	1125.40	1158.70	1122.00	1158.70	1113.20	1110.30
	4	1116.80	1153.10	1144.40	1153.10	1144.40	1153.10	1144.40	1153.10	1144.40	1128.20	1128.20
	5	1073.00	1100.00	1097.80	1100.00	1097.80	1100.00	1097.80	1100.00	1097.80	1089.70	1073.00
	6	1147.40	1174.20	1148.00	1174.20	1148.00	1174.20	1148.00	1174.20	1148.00	1147.40	1147.40
	7	1076.80	1113.00	1078.60	1113.00	1078.60	1113.00	1078.60	1110.70	1078.60	1076.80	1076.80
	8	1131.70	1177.10	1143.40	1177.10	1143.40	1177.10	1143.40	1140.80	1143.40	1131.70	1131.70
	9	1081.40	1159.40	1081.80	1159.40	1081.80	1159.40	1081.80	1122.60	1081.80	1082.20	1081.40
	10	1123.80	1172.50	1133.70	1172.50	1133.70	1172.50	1133.70	1153.70	1133.70	1125.10	1123.80

Table 9

Detailed results for ILS and ITS with 200 iterations (second part).

$ V_c $	Id.	Best	Initial solution		ILS heuristic		ILS exact		ITS heuristic		ITS exact	
			1 dir.	2 dir.	1 dir. 200 iter.	2 dir. 200 iter.	1 dir. 200 iter.	2 dir. 200 iter.	1 dir. 200 iter.	2 dir. 200 iter.	1 dir. 200 iter.	2 dir. 200 iter.
120	1	1472.49	1575.08	1526.53	1575.08	1526.53	1575.08	1526.53	1575.08	1526.53	1489.13	1490.64
	2	1482.29	1540.24	1593.14	1540.24	1593.14	1540.24	1593.14	1540.24	1593.14	1522.45	1565.99
	3	1510.05	1550.10	1676.84	1550.10	1676.84	1550.10	1676.84	1550.10	1676.84	1511.74	1512.77
	4	1563.50	1664.65	1661.45	1664.65	1661.45	1664.65	1661.45	1664.65	1661.45	1610.99	1649.77
	5	1457.05	1542.96	1599.23	1542.96	1599.23	1542.96	1599.23	1542.96	1599.23	1507.11	1475.97
	6	1546.25	1622.27	1670.92	1622.27	1670.92	1622.27	1670.92	1622.27	1670.92	1595.49	1595.10
	7	1557.27	1648.08	1610.88	1648.08	1610.88	1648.08	1610.88	1648.08	1610.88	1566.94	1572.40
	8	1524.07	1620.30	1599.61	1620.30	1599.61	1620.30	1599.61	1620.30	1599.61	1524.30	1526.81
	9	1547.95	1645.21	1620.27	1645.21	1620.27	1645.21	1620.27	1645.21	1620.27	1569.72	1564.53
	10	1573.87	1667.93	1622.36	1667.93	1622.36	1667.93	1622.36	1667.93	1622.36	1609.32	1574.02
140	1	1575.20	1654.40	1668.04	1654.40	1668.04	1654.40	1668.04	1654.40	1668.04	1598.20	1595.92
	2	1605.80	1670.76	1709.20	1670.76	1709.20	1670.76	1709.20	1670.76	1709.20	1638.14	1635.54
	3	1583.68	1648.92	1751.46	1648.92	1751.46	1648.92	1751.46	1634.40	1751.46	1608.02	1618.08
	4	1712.66	1787.94	1760.56	1787.94	1760.56	1787.94	1760.56	1774.64	1760.56	1736.94	1736.86
	5	1547.40	1633.14	1610.50	1633.14	1610.50	1633.14	1610.50	1633.14	1610.50	1580.56	1588.82
	6	1662.28	1759.38	1741.18	1759.38	1741.18	1759.38	1741.18	1759.38	1741.18	1662.82	1682.78
	7	1664.42	1749.36	1691.36	1749.36	1691.36	1749.36	1691.36	1719.60	1691.36	1670.46	1670.34
	8	1646.08	1741.96	1698.12	1741.96	1698.12	1741.96	1698.12	1705.24	1698.12	1646.08	1646.08
	9	1629.10	1700.02	1730.10	1700.02	1730.10	1700.02	1730.10	1700.02	1730.10	1670.50	1655.68
	10	1693.82	1717.34	1777.14	1717.34	1777.14	1717.34	1777.14	1717.34	1777.14	1693.82	1699.38
160	1	1741.75	1845.00	1836.63	1845.00	1836.63	1845.00	1836.63	1845.00	1836.63	1756.13	1756.13
	2	1773.63	1820.50	1856.75	1820.50	1856.75	1820.50	1856.75	1820.50	1856.75	1792.25	1791.88
	3	1690.88	1748.38	1828.25	1748.38	1828.25	1748.38	1828.25	1748.38	1828.25	1740.00	1740.00
	4	1858.13	1964.75	1962.63	1964.75	1962.63	1964.75	1962.63	1964.75	1962.63	1870.63	1909.25
	5	1667.75	1765.13	1756.75	1765.13	1756.75	1765.13	1756.75	1765.13	1756.75	1712.00	1711.75
	6	1813.00	1865.88	1844.50	1865.88	1844.50	1865.88	1844.50	1865.88	1844.50	1828.25	1828.25
	7	1774.25	1835.38	1871.00	1835.38	1871.00	1835.38	1871.00	1835.38	1871.00	1811.00	1866.88
	8	1770.75	1825.25	1852.88	1825.25	1852.88	1825.25	1852.88	1825.25	1852.88	1781.38	1852.88
	9	1800.88	1909.50	1895.25	1909.50	1895.25	1909.50	1895.25	1893.50	1895.25	1814.00	1887.38
	10	1764.38	1860.13	1809.38	1860.13	1809.38	1860.13	1809.38	1860.13	1809.38	1768.50	1785.50
180	1	1854.21	1936.01	1937.75	1936.01	1937.75	1936.01	1937.75	1936.01	1937.75	1865.81	1868.70
	2	1863.32	1959.69	1927.48	1959.69	1927.48	1959.69	1927.48	1959.69	1927.48	1880.28	1882.63
	3	1858.41	1947.95	1889.87	1947.95	1889.87	1947.95	1889.87	1926.52	1889.87	1874.71	1889.87
	4	1988.19	2067.97	2059.51	2067.97	2059.51	2067.97	2059.51	2067.97	2059.51	2022.70	2024.77
	5	1795.81	1869.75	1897.19	1869.75	1897.19	1869.75	1897.19	1869.75	1897.19	1802.33	1892.16
	6	1817.47	1873.17	1938.34	1873.17	1938.34	1873.17	1938.34	1873.17	1938.34	1845.49	1890.69
	7	1868.04	1929.56	1948.30	1929.56	1948.30	1929.56	1948.30	1929.56	1948.30	1874.29	1948.30
	8	1883.40	1964.13	1940.26	1964.13	1940.26	1964.13	1940.26	1930.46	1940.26	1895.99	1925.33
	9	1931.44	2004.60	2018.87	2004.60	2018.87	2004.60	2018.87	2004.60	2018.87	1940.08	1988.83
	10	1852.51	1923.11	1896.85	1923.11	1896.85	1923.11	1896.85	1923.11	1896.85	1872.85	1872.43
200	1	1976.30	2060.60	2065.70	2060.60	2065.70	2060.60	2065.70	2060.60	2065.70	2031.90	2065.70
	2	1982.00	2074.90	2077.60	2074.90	2077.60	2074.90	2077.60	2074.90	2077.60	2017.10	2077.60
	3	1976.70	2037.10	2090.90	2037.10	2090.90	2037.10	2090.90	2037.10	2090.90	1991.20	2090.10
	4	2119.50	2220.30	2211.90	2220.30	2211.90	2220.30	2211.90	2220.30	2211.90	2188.10	2211.90
	5	1905.60	1974.20	1986.80	1974.20	1986.80	1974.20	1986.80	1974.20	1986.80	1919.80	1986.80
	6	2011.10	2042.30	2088.00	2042.30	2088.00	2042.30	2088.00	2042.30	2088.00	2012.70	2084.60
	7	1983.00	2049.20	2048.90	2049.20	2048.90	2049.20	2048.90	2049.20	2048.90	1995.20	2048.90
	8	2000.30	2104.00	2096.30	2104.00	2096.30	2104.00	2096.30	2104.00	2096.30	2057.90	2096.30
	9	2052.70	2131.60	2141.10	2131.60	2141.10	2131.60	2141.10	2131.60	2141.10	2088.90	2141.10
	10	1977.90	2009.40	2066.30	2009.40	2066.30	2009.40	2066.30	2009.40	2066.30	1988.30	2066.30
Time (s)					113.68	113.68	1002.61	1002.46	57.47	57.44	471.79	471.93

results for ILS-1dir, ILS-2dir, ITS-1dir, and ITS-2dir. However, we have excluded the results with 50 and 100 iterations, for the sake of brevity.

References

- [1] Battarra M, Erdoğan G, Laporte G, Vigo D. The traveling salesman problem with pickups, deliveries, and handling costs. *Transportation Science* 2010;44:383–99.
- [2] Gribkovskaia I, Laporte G. One-to-many-to-one single vehicle pickup and delivery problems. In: Golden BL, Raghavan S, Wasil EA, editors. *The vehicle routing problem: latest advances and new challenges*. New York: Springer; 2008. p. 359–77.
- [3] Chambers PG, Grandin T. Transport of livestock, guidelines for humane handling, transport and slaughter of livestock. Food and Agriculture Organization of the United Nations, RAP Publication; 2001. p. 33–48.
- [4] Gribkovskaia I, Halskau Ø, Laporte G, Vlček M. General solutions to the single vehicle routing problem with pickups and deliveries. *European Journal of Operational Research* 2007;180:568–84.
- [5] Hoff A, Gribkovskaia I, Laporte G, Løkketangen A. Lasso solution strategies for the vehicle routing problem with pickups and deliveries. *European Journal of Operational Research* 2009;192:755–66.
- [6] Dell'Amico M, Falavigna S, Iori M. Optimization of a real-world auto-carrier transportation problem. Technical report, DISMI, University of Modena and Reggio Emilia; 2011.
- [7] Carrabs F, Cordeau J-F, Laporte G. Variable neighborhood search for the pickup and delivery traveling salesman problem with LIFO loading. *INFORMS Journal on Computing* 2007;19:618–32.

- [8] Cordeau J-F, Iori M, Laporte G, Salazar-González J-J. A branch-and-cut algorithm for the pickup and delivery traveling salesman problem with LIFO loading. *Networks* 2010;55:46–59.
- [9] Erdoğan G, Cordeau J-F, Laporte G. The pickup and delivery traveling salesman problem with first-in-first-out loading. *Computers & Operations Research* 2009;36:1800–8.
- [10] Petersen HL, Madsen OBG. The double travelling salesman problem with multiple stacks—Formulation and heuristic solution approaches. *European Journal of Operational Research* 2009;198:139–47.
- [11] Petersen HL, Archetti C, Speranza MG. Exact solutions to the double travelling salesman problem with multiple stacks. *Networks* 2010;56:229–43.
- [12] Iori M, Salazar-González J-J, Vigo D. An exact algorithm for the capacitated vehicle routing problem with two-dimensional loading constraints. *Transportation Science* 2007;41:253–64.
- [13] Iori M, Martello S. Routing problems with loading constraints. *TOP* 2010;18:4–27.
- [14] Laporte G. Comments on: Routing problems with loading constraints. *TOP* 2010;18:28–30.
- [15] Pacheco JA. Heurístico para los problemas de rutas con carga y descarga en sistemas LIFO. *Questiio* 1997;21:153–75.
- [16] Glover F. Future paths for integer programming and links to artificial intelligence. *Computers & Operations Research* 1986;5:533–49.
- [17] Lourenço HR, Martin OC, Stützle T. Iterated local search. In: Glover F, Kochenberger GA, editors. *Handbook of metaheuristics*. Boston: Kluwer; 2003. p. 321–53.
- [18] Misevičius A. Using iterated tabu search for the traveling salesman problem. *Information Technology and Control* 2004;3:29–40.
- [19] Mosheiov G. The travelling salesman problem with pick-up and delivery. *European Journal of Operational Research* 1994;79:299–310.
- [20] Gendreau M, Laporte G, Vigo D. Heuristics for the traveling salesman problem with pickup and delivery. *Computers & Operations Research* 1999;26:699–714.